

Intrusion Detection for Internet of Things: An Anchor Graph Clustering Approach

Yixuan Wu[✉], Long Zhang[✉], Lin Yang[✉], Feng Yang, Linru Ma[✉], Zhoumin Lu[✉],
and Wen Jiang[✉], *Member, IEEE*

Abstract—Intrusion detection systems are a crucial technique for securing the Internet of Things (IoT) from malicious attacks. Additionally, due to the continuous emergence of new vulnerabilities and unknown attack types, only a small number of attack samples in the IoT environments can be captured for analysis. In this work, we introduce an anchor graph clustering (AGC) method for intrusion detection to address the challenge of limited labeled samples in the IoT environments. AGC initially transforms the raw data into the embedding space to obtain more representative anchors. Then, AGC unifies anchor graph construction, anchor graph learning, and graph clustering into a unified framework, solving the resulting optimization problem through an iterative solution algorithm. Finally, AGC leverages the powerful analytical capabilities of graph learning to achieve fine-grained classification of low-quality labels. Experimental results on both real and synthetic datasets confirm that AGC can identify intrusions with high precision, while also being time-efficient in detection.

Index Terms—Internet of Things, intrusion detection, graph embedding, anchor graph clustering, anchor graph construction.

I. INTRODUCTION

THE production and use of smart devices has significantly expanded the reach of the Internet of Things (IoT). Today, IoT is widely applied in various sectors, such as public safety, smart grids and intelligent manufacturing, and has become an integral part of digital transformation [1]. Driven by innovation in the commercial and industrial sectors, the influx of large amounts of data has increased the burden on IoT devices for interaction, storage and analysis [2]. In addition, the overall performance and security of IoT devices is generally lower than that of traditional computing devices, making IoT systems particularly vulnerable to malicious attacks [3].

Received 1 July 2024; revised 10 December 2024; accepted 1 February 2025. Date of publication 05 February 2025; date of current version 18 February 2025. This work was supported by the National Natural Science Foundation of China under Grant 62171378. The associate editor coordinating the review of this article and approving it for publication was Dr. Ashok Kumar Das. (*Corresponding author: Lin Yang.*)

Yixuan Wu and Wen Jiang are with the School of Electronics and Information, Northwestern Polytechnical University, Xi'an 710072, China (e-mail: wuyixuan1@mail.nwpu.edu.cn; jiangwen@nwpu.edu.cn).

Long Zhang, Feng Yang, and Linru Ma are with the Institute of Systems Engineering, AMS, Beijing 100141, China (e-mail: zhanglong10@nudt.edu.cn; yfzy1221@163.com; malinru@163.com).

Lin Yang is with the School of Electronics and Information, Northwestern Polytechnical University, Xi'an 710072, China, and also with the Institute of Systems Engineering, AMS, Beijing 100141, China (e-mail: yanglin61s@126.com).

Zhoumin Lu is with the School of Computer Science, Northwestern Polytechnical University, Xi'an 710072, China (e-mail: walker.zhoumin.lu@gmail.com).

Digital Object Identifier 10.1109/TIFS.2025.3539100

Network security is essential for the widespread adoption of the IoT. The development of intrusion detection methods suitable for the IoT has become a major research focus in this area [4]. Intrusion detection systems (IDS), which actively collect network traffic and can perform real-time security detection without affecting the operation of IoT devices, are one of the primary technologies for IoT security [3]. Recent research in intrusion detection has focused on the use of machine learning (ML) models because of their ability to automatically learn from heterogeneous data [5]. Despite some research focused on using artificial intelligence (AI) techniques to detect anomalies in the IoT, several proposed methods lack transparency and network resilience [6]. Additionally, obtaining a rich set of high-quality labels in the IoT is challenging. Unlike labelling images, investigating new attacks requires extensive expertise [7]. Consequently, in the IoT, only a small portion of samples can be effectively labeled [7]. Low-quality labeled data can degrade machine learning performance and be detrimental to practical applications that rely on label-based operations [8], [9]. Therefore, customized intrusion detection as a second line of defense has become imperative for the IoT.

Compared to traditional networks, the various components and network layers of the IoT are more vulnerable to intrusion threats. In addition, IoT nodes need to achieve efficient intrusion detection despite low computational power and low-quality data [10]. In this paper, our goal is to achieve intrusion detection with low-quality labels using anchor graph clustering (AGC) approach. The aim is to uncover the clustering structure within the data to obtain high-quality anchors in the IoT [11].

The robust data analysis capability of graph clustering enables it to extract normal user behavior patterns from extensive training data [12], [13]. AGC utilizes a high-quality anchor matrix to achieve clustering results, thereby providing competitive anomaly detection performance. As shown in Fig. 1, first, we determine the number of types of low-quality data, as identifying the correct number of types can improve AGC's clustering accuracy. Next, based on the identified number of types, we perform anchor selection and anchor graph construction jointly in an unsupervised manner. These two processes can coordinate with each other to enhance clustering quality. Finally, the anchors generated by AGC can facilitate fine-grained data classification and subsequent intrusion detection.

Given the redundancy and corruption of IoT data, this paper maps the raw data to an embedding space to obtain more

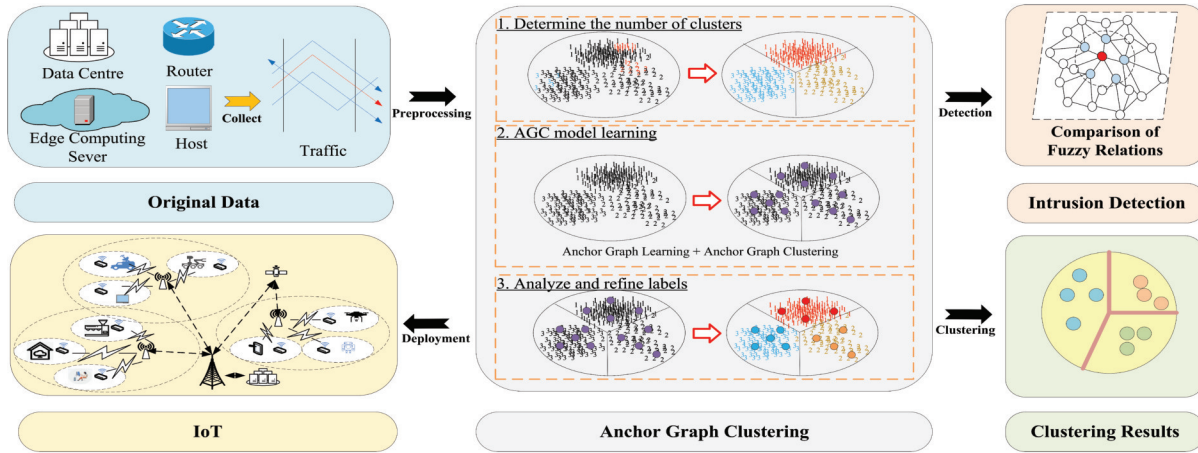


Fig. 1. Anchor graph clustering approach flowchart. Starting with the original low-quality labeled data, we first redefine the number of data categories. Next, the data is projected into an embedding space to enhance anchor quality and improve clustering performance. Finally, the original labels are refined based on the resulting outcomes.

representative anchors. These anchors help reduce time overhead and computational complexity. Meanwhile, to explore more discriminative information and address the problem of missing labeled data, this paper proposes dynamically guided anchor graph learning. In addition, this paper unifies anchor graph construction, graph learning, and graph clustering into a single framework. AGC enforces k connected constraints on anchor graphs to directly generate clustering results without any subsequent processing methods. Finally, AGC utilizes a graph clustering approach to achieve intrusion detection by comparing the fuzzy relationships between anomalous data and anchors. Applying the AGC model trained on low-quality labeled datasets to real IoT environments can effectively resist intrusion attacks, achieving real-time intrusion detection with low overhead. In summary, the key contributions of AGC are as follows:

- We propose AGC to address the problem of low-quality data labels, a common challenge in IoT intrusion detection based on ML. AGC can detect anomalies in both labeled and unlabeled scenarios. It reduces the time-consuming and error-prone problems associated with acquiring labeled data.
- We propose a novel embedding space dynamic anchor graph learning method. AGC integrates embedding learning, anchor graph construction, graph learning, and graph clustering into a unified framework.
- The embedding space learning method proposed in this paper can filter out unfavorable information while reducing computational dimensions. Furthermore, this method can be extended to multi-view learning, such as integrating traffic, logs, and other multi-view information, and unifies the dimension of each view through embedding space learning.
- In the experiment, we compared the detection and classification performance of AGC. The results showed that it has certain advantages over existing methods.

The composition of this paper is structured as follows. In Section I, we describe the problems of existing IoT intrusion

detection and the innovations of AGC. Then, in Section II, the work related to intrusion detection and graph clustering is briefly presented. The specifics of AGC and its optimization are detailed in Section III and Section IV, respectively. In Section V, comprehensive experiments are conducted to validate the effectiveness of AGC. Finally, the conclusions are summarized in Section VI.

II. RELATED WORK

A. Intrusion Detection Technologies

In the IoT architectures, IDS typically operate at the network layer, employing two primary detection methods: signature-based and anomaly-based. Based on anomaly-based intrusion detection techniques, there are three main types: unsupervised learning, semi-supervised learning, and supervised learning [14], [15]. Unsupervised learning aims to preserve as much original information as possible, semi-supervised learning uses a small amount of labelled data to achieve better performance, while supervised learning focuses primarily on information related to the supervised task.

AI, such as ML, has been used as a powerful tool to address the IoT security due to its ability to handle data heterogeneity and processing speed [16]. To address the need for lightweight, fast, and flexible security solutions in IoT applications and networks, Latif et al. [17] proposed a lightweight dense random neural network for IoT devices. This model built separate classification models for nine different IoT attacks, and used a multi-layer structure based on dense clusters to achieve improved detection accuracy. Similarly, Kamaldeep et al. [18] developed an IDS called IoT-Sentry, a cross-layer IDS capable of detecting five different attacks with zero additional overhead through integrated learning. Although these models can be applied to IoT devices, their protection against only a limited number of attacks restricts their widespread use.

In response to the evolving nature of network attack behaviors, Tellache et al. [10] proposed a novel multi-agent reinforcement learning architecture for efficient network intrusion detection. To enhance the model's capabilities, this

framework improves the deep Q-network algorithm with a weighted mean square error loss function and introduces a flexible architecture to accommodate changes in existing attack patterns. Meanwhile, recent IDS advancements have leveraged federated learning methods to enable distributed and privacy-preserving intrusion detection. For example, Korba et al. [19] utilized blockchain technology to promote trustworthy and decentralized federated learning within IDS frameworks. They proposed an innovative Zero-X security framework that effectively detects 0-day and N-day attacks. This framework achieves its goal by combining deep neural networks with open-set recognition, enabling effective detection of 0-day and N-day attacks in the Internet of Vehicles domain. The above methods are typically passive, responding to specific patterns or observed anomalies. Unlike these methods, Diaf et al. [20] proposed an active approach that can predict an attack before malicious activity causes damage. This approach combines a large language model (LLM) with a long short-term memory (LSTM) network, leveraging the LLM for traffic prediction and subsequently using the LSTM to detect predicted traffic, ultimately identifying anomalous data.

Intrusion detection is a critical problem in the current field of IoT research, with studies already conducted in areas such as open-set learning, reinforcement learning, and generative AI. Based on an analysis of related literature, it can be concluded that existing IDS models primarily aim for high accuracy and applicability on IoT devices by combining multiple ML techniques. However, achieving high detection accuracy with simple ML models remains a significant challenge in the absence of labeled data. To address these challenges, this paper proposes an IDS based on AGC, which accomplishes data dimension reduction and clustering in a single operation.

B. Clustering-Based Intrusion Detection

Clustering is an unsupervised technique that primarily uses similarity to group data. For example, Hu et al. [21] proposed an intrusion detection method based on a multi-core clustering algorithm to address the issue of missing traffic features in IoT environments. Unlike traditional methods that use zero or mean value imputation, they calculate the missing traffic features using similarity measures, and then apply a multi-core K-means clustering approach for intrusion detection. The traditional K-means algorithm is sensitive to the initial values of the cluster centers, and its accuracy can be easily affected by noisy data. To avoid these issues, Iam-On [22] used different consensus clustering techniques to analyze noisy data, implementing data analysis based on K-means clustering. Similarly, Cerdà-Alabern et al. [23] employed four unsupervised machine learning methods based on different principles for anomaly detection. By combining K-means with other methods, they found that, with appropriate adjustments, all methods could effectively detect anomalies. Most of the aforementioned methods combine K-means with other classification algorithms to improve detection rates. However, a drawback of K-means is that the number of clusters, k , must be specified in advance, which is neither a simple nor intuitive task. The correct selection of k is often unclear, as it heavily depends on the

shape and scale of the data distribution. Additionally, because the initial centroids are often randomly chosen, the algorithm is prone to getting stuck in local optima.

C. Anchor-Based Clustering

Graph clustering algorithms have found success in multiple domains, including image segmentation, text mining, anomaly detection, and speech recognition [24], [25]. Existing graph clustering methods, while effective at improving clustering performance, suffer from high computational and spatial complexity, making them unsuitable for the IoT. Anchor graph have emerged as a recognized solution to these challenges [26]. Therefore, researchers have introduced the anchor graph paradigm for large-scale data clustering. This approach constructs an anchor graph, which approximates the distribution of data points by using a subset of instances as anchors, rather than the entire dataset.

For instance, Li et al. [27] effectively minimized the costly time overhead by designing a simple and initialization independent strategy for selecting anchor points. Following their selection of anchor points, they incorporate k connected constraints to refine the clustering process. On the other hand, Kang et al. [28] aimed to secure more representative anchor points by employing the K-means method. Subsequently, they refined the final clustering process through interactive learning involving bipartite graphs and cluster indicator matrix.

However, these methods still have certain drawbacks: (1) The quality of anchor points significantly influences the final model results [29], yet anchor points are typically selected using heuristic methods that can be highly randomized. (2) Anchor point selection and anchor matrix learning are often treated as separate, independent steps, potentially obscuring the direct impact of anchor points on the final results. In contrast to deep learning (DL), traditional detection methods typically offer advantages such as lower complexity and reduced computational resource consumption. This paper aims to exploit the latent information in IoT network data features by proposing an intrusion detection model based on AGC.

D. Anchor Graph Clustering Vs K-Means

The K-means technique is one of the most popular clustering analysis methods, aiming to divide n data objects into k clusters, where each data object is assigned to the cluster with the closest mean [21]. K-means is a distance-based clustering technique that uses the Euclidean metric as a similarity measure, with the number of clusters k pre-specified by the user. Despite being a widely useful clustering algorithm, K-means has certain limitations. It is sensitive to noise and outliers, requires prior knowledge of the number of clusters k , and its objective function is non-convex, which may lead to local optima. Unfortunately, this means the algorithm is prone to getting trapped in local minima. The efficiency of the K-means algorithm heavily depends on the choice of the initial random centroids, and it may not perform well for clusters with varying shapes.

Anchors are a set of points significantly fewer in number than the original data, which can roughly represent the

structure of the original data [27]. Compared to full-size data computations, anchor graph calculations can significantly reduce storage and computational demands. Constructing an anchor graph involves several steps: first, representative anchors are generated based on a certain strategy, and then the anchor graph is built by calculating the similarity between data points and anchors. Each data point is then represented as a linear combination of the anchors, minimizing the distance between the data point and the anchors, effectively mapping the data points into a lower-dimensional space. To ensure that anchors provide high discriminative power when representing data points, AGC usually applies orthogonal constraints on the anchor representation matrix. This constraint enforces independence among the anchor representations, allowing samples of the same class to more easily fall into the same subspace, while remaining linearly independent from samples in other subspaces. Finally, spectral clustering is performed on the anchor graph, constructing a low-dimensional representation through the anchors and converting the problem to spectral clustering on the anchors, thereby avoiding full matrix decomposition of the original data and obtaining the final clustering result.

The difference between K-means and AGC lies in what they aim to identify: K-means finds the centroid of each cluster (the mean of the samples within the cluster), where the distance of samples to their own cluster's centroid is smaller than to the centroids of other clusters. In contrast, AGC identifies orthogonally constrained anchors, where similar samples fall into the same subspace and can be linearly reconstructed by samples within that subspace, remaining linearly independent from samples in other subspaces. Additionally, the number of anchors is usually required to exceed the actual number of categories k , providing finer granularity and greater discriminative power. Through extensions, AGC can also support dimensionality reduction and adapt to multi-view data.

III. METHOD

A. Motivation

The goal of this paper is to develop a real-time network intrusion detection method for identifying malicious data in IoT networks. By fully utilizing available labeled data records, we aim to enhance the detection capabilities of IoT intrusion detection models. We focus on the detection of active attacks, such as injection attacks, which are constructed using methods like traffic analysis. Existing research indicates that attackers perform reconnaissance to probe the victim for sensitive information [30]. Therefore, by observing the overall network state through methods such as traffic analysis, it is possible to detect anomalous behavior indicative of attacks. In addition, the proposed method can be extended to a multi-view consistent detection approach, enabling consistent learning and detection using data such as traffic and logs.

In real-world IoT environments, the lack of fine-grained analysis often leads to data from multiple categories being misclassified into one, or certain categories not being classified at all. These low-quality labels tend to perform poorly in IoT-oriented intrusion detection. Fine-grained labels are critical

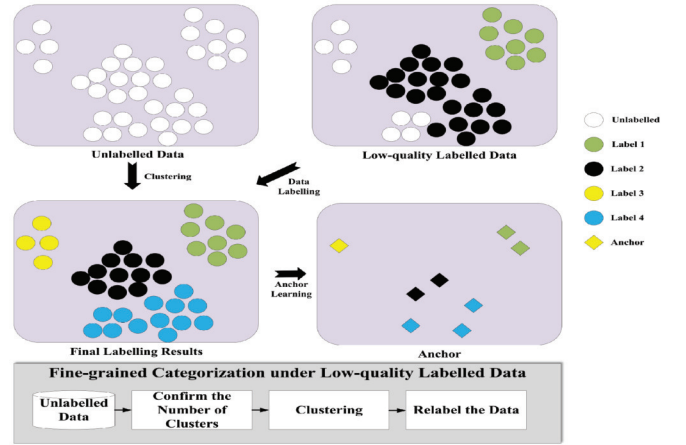


Fig. 2. Fine-grained labels under low-quality data.

TABLE I
NOTATIONS AND DEFINITIONS

Notation	Definition
n	The number of samples
d	The dimension of the samples
m	The dimension of the embedding space
a	The number of anchors
k	The number of clusters
$\mathbf{1}$	The all elements are 1 in the column vector $\mathbf{1}$
$\bar{R}_i, \underline{R}_i$	The upper and lower approximations of a data point x_i
$\mathfrak{R}$	The adaptive fuzzy relationship
$\mathbf{X} \in \mathbb{R}^{d \times n}$	Sample Data Matrix
$\mathbf{W} \in \mathbb{R}^{d \times m}$	Projection matrix
$\mathbf{A} \in \mathbb{R}^{m \times a}$	Anchor matrix
$\mathbf{Z} \in \mathbb{R}^{a \times n}$	Anchor graph
$\mathbf{O}$	Augmented graph of $\mathbf{Z}$
$\mathbf{D}_O$	Degree matrix
$\tilde{\mathbf{L}}_O$	Normalized Laplacian matrix

for effective intrusion detection [31]. Therefore, we use AGC to learn from the limited low-quality labels, reflect the true distribution of the raw data, and further use anchor graphs for real-time intrusion detection. As illustrated in Fig. 2, we use the clustering power of AGC to reassign the label distribution of the data. In the AGC method, we use reduced-dimensional hidden space to represent the data distribution. The dimensionality reduction method can filter out redundant information from the original space, allowing us to perform more accurate clustering. We then proceed with anchor graph learning to obtain high quality anchor for real-time intrusion detection.

In summary, for intrusion detection in IoT environments with low-quality labels, we first need to determine the number of clusters in the raw data. Then, we use AGC for fine-grained clustering and establish anchor points. Finally, we develop a real-time intrusion detection method based on these anchors. The most important notations are summarized in Table I.

B. Number of Clusters

The final clustering performance of AGC depends on the choice of the number of clusters k . In an unsupervised environment, clustering cannot proceed until the value of k

is determined. Let the upper approximation $\bar{R}_i$ be the average distance between sample $x_i \in S_l$ and all other data points in the same cluster $l \in \{1, \dots, k\}$, where S_l represents the set of data points in cluster l , the number of samples is n , the number of clusters is k . The lower approximation $\underline{R}_i$ is the minimum value among the average distances between x_i and all data points in other clusters $S_j (j \neq l)$. Thus, the data point x_i can be fuzzy approximated as follows:

$$\begin{cases} FA_i = \frac{\underline{R}_i - \bar{R}_i}{\max\{\underline{R}_i, \bar{R}_i\}}, & \text{if } |S_j| > 1 \\ FA_i = 0, & \text{if } |S_j| = 1. \end{cases} \quad (1)$$

Given the fuzzy approximation FA_i for each sample, the combined fuzzy approximation $FA(k)$ of k clusters is defined as the maximum $\max\left\{\sum_{i=1}^k FA(l)\right\}$ among all clusters, where $FA(l)$ is the average of the FA_i values for all samples within cluster l . The $FA(k)$ can be used to determine the most appropriate number of clusters k by performing multiple AGC tests. With the obtained number of clusters k , we can further use AGC to learn the underlying structure of the raw data.

C. Anchor Graph Clustering

This paper extracts the underlying data distribution using anchor points to reduce the uncertainty caused by the original low quality labels in the IoT. As can be seen from the following equation, anchor graph learning approximates the overall distribution of the raw data by using a subset of representative raw samples as anchor points [29].

$$\min_{Z \geq 0, Z^T \mathbf{1} = \mathbf{1}} \|X - AZ\|_F^2, \quad (2)$$

where $X \in \mathbb{R}^{d \times n}$ is data matrix, where d represents the dimension of the sample, and there are n data points $\{x_1, \dots, x_n\}$. $A \in \mathbb{R}^{d \times a}$ denotes the anchor matrix, a is the number of anchors. $Z \in \mathbb{R}^{a \times n}$ represents the anchor graph, used to represent the relationship between the raw data and anchor matrix. The limitations of IoT devices prevent large-scale computations from being performed on them. Therefore, by selecting high-quality anchor for comparison calculations, we can reduce the computational load and enable real-time operations on IoT devices.

However, the raw high-dimensional data collected in IoT networks inevitably contains redundant information and corrupted features, which can negatively affect the detection quality of anchor graph learning methods. Although feature selection methods can remove redundant information, the absence of certain features can also affect the final results. By using feature dimensionality reduction, we can not only reduce the influence of redundant information, but also allow the raw data to be clustered into fine-grained clusters in a low-dimensional space. In addition, existing research widely acknowledges that anchor quality plays a critical role in clustering performance. Adaptive anchor optimization has been proven effective in clustering. However, traditional anchor selection strategies are static, involving random sampling or heuristic-based selection of anchors before clustering. These static methods suffer from two main limitations: the high

randomness of sampling and the adverse effects of noise, leading to selected anchors that fail to accurately reflect the true data distribution. Moreover, anchor selection and clustering are treated as two independent steps, resulting in anchors that may not be suitable for clustering. To address these issues, we adaptively learn a set of high-quality anchors in a clean embedding space. By employing an optimal learning approach, anchor learning and anchor graph construction are unified into a consistent process, where anchors are iteratively updated through alternating optimization. The dynamically learned anchors retain more representative information, enabling the relationships between all data points to be represented via the relationships between anchors and data points. Furthermore, the anchor graph effectively captures the true structural information required for clustering.

Embedding learning maps data from the primal space to an embedded space, reducing dimension and significantly avoiding the impact of unfavorable noise. Based on these considerations, this paper uses embedding learning to derive a new representation for the original data, and then learns anchors in the embedding space instead of the primal space. In summary, this can be described as follows:

$$\begin{aligned} & \min_{W, A, Z} \|W^T X - AZ\|_F^2 \\ & \text{s.t. } W^T W = I_m, \quad A^T A = I_a, \\ & \quad Z \geq 0, \quad Z^T \mathbf{1} = \mathbf{1}, \end{aligned} \quad (3)$$

where $W \in \mathbb{R}^{d \times m}$ is the sample projection matrix and $m (m < d)$ is the dimension of the embedding space. It is important to note that the orthogonal condition $W^T W = I_m$ not only improves discrimination, but also prevents trivial solutions. $A \in \mathbb{R}^{m \times a}$ is the anchor matrix in the embedding space. $A^T A = I_a$ constrains the anchor matrix to make them more distinguishable. Therefore, the anchor matrix more accurately models the distribution of all data in the embedding space. In addition, the learned high-quality anchor graph facilitates data clustering, while the anchor matrix is used for intrusion detection. To avoid the uncertainty introduced by subsequent processing steps such as spectral rotation [32] or K-means [33], this paper directly obtains results by applying rank constraints to the anchor graph. Thus, the final objective function OAGC can be expressed as follows:

$$\begin{aligned} & \min_{W, A, Z} \|W^T X - AZ\|_F^2 \\ & \text{s.t. } W^T W = I_m, \quad A^T A = I_a, \quad Z \geq 0, \quad Z^T \mathbf{1} = \mathbf{1}, \\ & \quad \text{rank}(\tilde{L}_O) = n + a - k, \end{aligned} \quad (4)$$

where $\tilde{L}_O = I - D_O^{-1/2} O D_O^{-1/2}$ is the normalized Laplacian matrix of O , and the degree matrix D_O is a diagonal matrix associated with O , k represents the number of sample clusters. The diagonal elements of D_O are computed from $D_O(i, i) = \sum_{j=1}^{n+m} O_{ij}$. O is the augmented graph of Z , defined as:

$$O = \begin{bmatrix} & Z^T \\ Z & \end{bmatrix} \in \mathbb{R}^{(n+a) \times (n+a)}. \quad (5)$$

Theorem 1 [27] shows that the Laplacian rank constraint on O ensures that O has k connected components, meaning

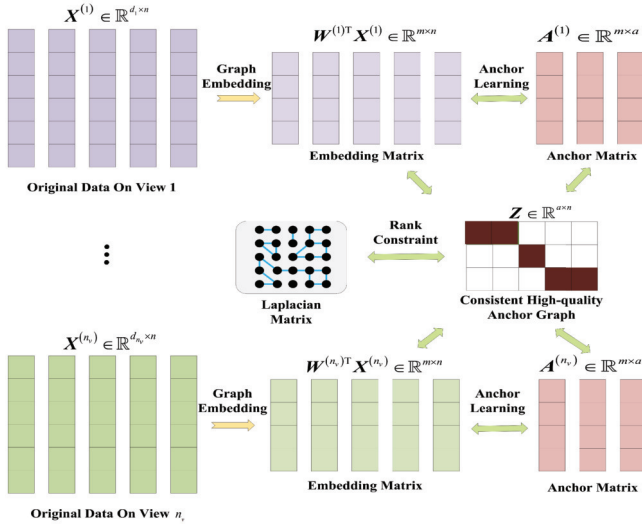


Fig. 3. Anchor graph clustering approach.

the data is divided into k classes. Due to the structure of Eq. (5), O is composed of Z . The fact that O has k connected components ensures that Z also has k connected components. Consequently, the final result can be directly expressed in terms of these connected components, allowing the clustering result to be generated by Z without any post-processing [27]. Thus, using (4), the anchor matrix is learned dynamically instead of relying on traditional static sampling.

Theorem 1: The number of zero eigenvalues of the normalized Laplacian matrix $\tilde{L}_O$ is equal to the number of connected components in the graph associated with O .

In this paper, we focus on the detection of active attacks constructed using methods such as traffic analysis. However, the entire IoT system includes different types of data, such as traffic and logs, each with different potential distributions and representations. To address these problems, our proposed OGC can be further extended to a multi-view framework. Optimally, each cluster should maintain consistent clustering centres across all perspectives. This helps to reduce discrepancies between the raw data and its underlying distribution, ensuring the accuracy of anomaly detection. The unified embedding space can be extended to multiple views, allowing all views to be reduced to consistent dimension. AGC achieves the generation of consistent high-quality anchor graph by dynamically learning the embedding data.

Given n samples in d_i dimensional space within the i -th view $X^{(i)} \in \mathbb{R}^{d_i \times n}$ ($i = 1, 2, \dots, n_v$), multi-view AGC is constructed as follows:

$$\begin{aligned} \min_{W^{(i)}, A^{(i)}, Z} \sum_{i=1}^{n_v} \|W^{(i)T} X^{(i)} - A^{(i)} Z\|_F^2 \\ \text{s.t. } W^{(i)T} W^{(i)} = I_m, A^{(i)T} A^{(i)} = I_a, \quad Z \geq 0, \\ Z^T \mathbf{1} = \mathbf{1}, \quad \text{rank}(\tilde{L}_O) = n + a - k, \end{aligned} \quad (6)$$

where n_v is the number of views, $W^{(i)} \in \mathbb{R}^{d_i \times m}$ is the projection matrix for the i -th view. $A^{(i)} \in \mathbb{R}^{m \times a}$ represents the anchor matrix of the i -th view. Z is consistent anchor graph.

As illustrated in Fig. 3, the original data is initially down-scaled through graph embedding learning to reduce redundant

information. Then, the refined graph data undergoes dynamic anchor graph learning. Finally, classification results are derived by applying k connected constraints to the anchor graph. In this study, the learned anchor matrix is used to construct an IDS. Compared to DL methods, AGC obtains detection results through fuzzy relationships, providing a basis for intrusion detection. In summary, the characteristics of AGC can be described as follows:

- **Embedding learning:** Filters redundant and corrupted information in a low-dimensional space to improve clustering accuracy.
- **Joint learning:** Combines embedding learning, anchor graph construction, anchor graph learning, and clustering into a unified framework for dynamic learning.
- **Computational efficiency:** Anchor graph learning reduces the overall computational complexity, and the iterative update mechanism facilitates efficient solution of the AGC problem.

D. Intrusion Detection

After obtaining the anchor points and optimal clustering structure of the original data through AGC, we re-label the data based on the original low-quality labels, for example, by subdividing the anomaly class into multiple categories. This subdivision can be directly derived through the analysis of the clustering results.

The primary technique for identifying anomalies is to designate points that are significantly away from the cluster center as anomalies [34], [35]. For the optimal anchor point A obtained through iterative updates, the difference between real-time network data points and the anchor point is calculated. Data points with differences exceeding a certain threshold from the anchor point are identified as anomalies, while those within the threshold range are classified as similar to the anchor point.

For new samples, detection can be achieved by comparing them with anchor points using a fuzzy relationship, thereby reducing the need to retrain the model. Given n_t data points in d_i dimensions in the i -th view $X_t^{(i)} \in \mathbb{R}^{d_i \times n_t}$ ($i = 1, 2, \dots, n_v$) collected by the IoT, the adaptive fuzzy relationship between a subsequent data point $x_t \in W^{(i)T} X_t^{(i)}$ on the embedding feature $c_j \in M$ ($j = 1, 2, \dots, m$) and the anchor matrix $x_e \in A^{(i)}$ is expressed as follows:

$$\mathcal{R}_{\{c_j\}}^{\rho_j}(x_t, x_e) = \begin{cases} 1 - \frac{|V_{c_j}(x_t) - V_{c_j}(x_e)|}{(1 + \text{std}_{\{c_j\}})}, & \text{if } |V_{c_j}(x_t) - V_{c_j}(x_e)| \leq \rho_j \\ 0, & \text{if } |V_{c_j}(x_t) - V_{c_j}(x_e)| > \rho_j, \end{cases} \quad (7)$$

where $\rho_j = \text{std}_{\{c_j\}}/\xi$ denotes the adaptive neighborhood in feature c_j , $\text{std}_{\{c_j\}}$ denotes the standard deviation on c_j , and the adaptive neighborhood parameter ξ can be adjusted according to different scenarios for multi-party validation. $V_{c_j}(x_t)$ denotes the value of x_t in feature c_j . As shown in (8), anomaly detection can be performed by comparing the adaptive fuzzy relationship between each data point and the anchor matrix.

$$P = \left\{ \mathcal{R}_{\{c_1\}}^{\rho_1}(x_t, x_e), \dots, \mathcal{R}_{\{c_m\}}^{\rho_m}(x_t, x_e) \right\}, \quad (8)$$

where P represents the relationship between the data point x_t and the anchor point x_e in terms of features. When P contains no zero values, x_t and x_e are strongly correlated; when x_e contains zero values, x_t and x_e are weakly correlated. If the data is strongly correlated, set x_t as the same type of data as the anchor point corresponding to $P' = \max_e \sum_{j=1}^m \mathfrak{R}_{\{c_j\}}^{\rho_j}(x_t, x_e) \in [0, m]$. If the data is weakly correlated, it needs to be determined through data comparison. The target anchor point for comparison is the one with the highest P' value. If the distance between the data point and the anchor point is less than the threshold $P'' = \max_{x_t \in S_l} \bar{R}_i$, the data point is defined as a new sample type. Otherwise, if the distance falls within the threshold, the data point is defined as a similar type to the target anchor point. Where the threshold P'' is established based on the upper approximation $\bar{R}_i$ derived from Eq. (1). The threshold P'' represents the maximum upper approximation distance between the original data within cluster l (the category of the target anchor point). The threshold can be set based on specific experimental conditions. For instance, when the original data is fully labeled, the corresponding thresholds for each anchor point can be adjusted to varying degrees according to the boundary ranges between different types of the original data. Our method relies on clustering during the learning process to uncover the intrinsic distribution of the samples and identify representative anchor points. The class represented by each anchor point is jointly determined by multiple labeled data points and the clustering results, making it less affected by low-quality labeled data.

E. Algorithm Description

The details of the AGC intrusion detection process are described as follows.

- **Preprocessing:** Data preprocessing plays a critical role in improving data quality for intrusion detection tasks. Feature standardisation and normalisation can be applied, where each feature is centred and rescaled to have a variance of 1. This preprocessing ensures that features are scaled to similar sizes, preventing any single feature from dominating.
- **Training:** Using (6), we can learn data distribution from the original data. Before training, algorithm parameters need to be initialized. A good initialization method can improve the accuracy of the model.
- **Detection:** For real-time intrusion detection, (8) can be used for analysis. For non-real-time data, (6) can be used directly to extract the underlying data distribution, reducing the uncertainty caused by insufficient original labels.

Overall, the input to AGC consists of a small fraction of labeled data, and the label quality is relatively low. AGC can output allocation results for all data, where data either forms new groups to represent unknown labels or clusters correctly under known labels. By accurately recovering the clustering structure in the input dataset, AGC provides much-needed support for generating high-quality labels. The core

Algorithm 1 Intrusion Detection Algorithm Based on AGC

Input: Training data $\{X^{(i)}\}_{i=1}^{n_v}$, test data $\{X_t^{(i)}\}_{i=1}^{n_v}$, m , and a ;
Outputs: Detecting results.

- 1: Data preprocessing;
- 2: The AGC model is established based on Eq. (6);
- 3: Calculate k by using (1);
- 4: The AGC model is trained using the obtained k value and the training data, resulting in the anchor matrix $A^{(i)}$ and the anchor graph Z ;
- 5: Re-label the low-quality data based on the clustering results;
- 6: Record the threshold $P'' = \max_{x_t \in S_l} \bar{R}_j$ for different clusters, where $l \in \{1, \dots, k\}$;
- 7: **for** each data x_t in $X_t^{(i)}$ **do**
- 8: **for** each data x_e in $A^{(i)}$ **do**
- 9: Calculate adaptive fuzzy relationship $\mathfrak{R}_{\{c_j\}}^{\rho_j}(x_t, x_e)$ by using (7), where $c_j \in M$ ($j = 1, 2, \dots, m$);
- 10: Calculate the feature relationship value P by using (8);
- 11: **if** P contains no zero values **then**
- 12: x_t is strongly correlated;
- 13: **else if** P contains zero values **then**
- 14: x_t is weakly correlated;
- 15: **end if**
- 16: **end for**
- 17: Calculate $P' = \max_e \sum_{j=1}^m \mathfrak{R}_{\{c_j\}}^{\rho_j}(x_t, x_e)$;
- 18: **if** x_t is strongly correlated **then**
- 19: Set x_t as the same type of data as the anchor point corresponding to P' ;
- 20: **else if** x_t is weakly correlated **then**
- 21: If the distance between x_t and the anchor point is less than the threshold P'' , x_t is defined as a new sample type. Otherwise, x_t is defined as a similar type to the target anchor point;
- 22: **end if**
- 23: **end for**
- 24: Repeat steps 4–23 to compute the multi-view data. When the results from different views are inconsistent, the one with the highest P' value is used as the reference;

idea of AGC is to fully utilize the limited number of labels, thereby achieving intrusion detection for the IoT. The overall performance of the AGC-based intrusion detection method is outlined in Algorithm 1.

IV. ALGORITHM OPTIMIZATION AND DESCRIPTION

This section focuses on solving the established AGC model and formulating the final algorithm. The (6) is a multi-view extension of the (4). Solving (6) is equivalent to solving the final unified model AGC. However, it should be noted that (6) is not jointly convex when all variables are updated simultaneously. Therefore, we approach solving (6) by using an iterative update mechanism. This process is divided into several steps, where each variable is updated sequentially while keeping the others fixed.

A. Update W

Fixing $A^{(i)}$, and Z , the optimization of $W^{(i)}$ in (6) can be expressed as follows:

$$\min_{W^{(i)T}W^{(i)}=I_m} \sum_{i=1}^{n_v} \|W^{(i)T}X^{(i)} - A^{(i)}Z\|_F^2. \quad (9)$$

As each $W^{(i)}$ is independent for each view, the problem is addressed by extending the Frobenius paradigm using the trace operator. Consequently, (9) can be converted into the following form:

$$\begin{aligned} & \min_{W^{(i)T}W^{(i)}=I_m} \sum_{i=1}^{n_v} \|W^{(i)T}X^{(i)} - A^{(i)}Z\|_F^2 \\ & \Leftrightarrow \max_{W^{(i)T}W^{(i)}=I_m} \text{Tr}(Z^T A^{(i)T} W^{(i)T} X^{(i)}) \\ & \quad + \max_{W^{(i)T}W^{(i)}=I_m} \text{Tr}(X^{(i)T} W^{(i)} A^{(i)} Z) \\ & \Leftrightarrow \max_{W^{(i)T}W^{(i)}=I_m} \text{Tr}(W^{(i)T} X^{(i)} Z^T A^{(i)T}) \\ & \Leftrightarrow \max_{W^{(i)T}W^{(i)}=I_m} \text{Tr}(W^{(i)T} P_W^{(i)}), \end{aligned} \quad (10)$$

where $P_W^{(i)} = X^{(i)} Z^T A^{(i)T}$. Assume that the matrix in (10) has the singular value decomposition (SVD) form as $P_W^{(i)} = U_W^{(i)} \Sigma_W^{(i)} V_W^{(i)T}$, with $U_W^{(i)} \in \mathbb{R}^{d_i \times m}$, $\Sigma_W^{(i)} \in \mathbb{R}^{m \times m}$, and $V_W^{(i)} \in \mathbb{R}^{m \times m}$. Equation (10) can be formulated as follows:

$$\begin{aligned} \text{Tr}(W^{(i)T} P_W^{(i)}) &= \text{Tr}(W^{(i)T} U_W^{(i)} \Sigma_W^{(i)} V_W^{(i)T}) \\ &= \text{Tr}(V_W^{(i)T} W^{(i)T} U_W^{(i)} \Sigma_W^{(i)}). \end{aligned} \quad (11)$$

Let $Q = V_W^{(i)T} W^{(i)T} U_W^{(i)}$, we have $QQ^T = I_m$. Consequently, we can select $\text{Tr}(V_W^{(i)T} W^{(i)T} U_W^{(i)} \Sigma_W^{(i)}) = \text{Tr}(Q \Sigma_W^{(i)}) \leq \sum_{i=1}^m \sigma_i$, where σ is the element in $\Sigma_W^{(i)}$. Therefore, in order to maximize the value of (10), the optimal solution is $W^{(i)} = U_W^{(i)} V_W^{(i)T}$.

B. Update A

Fixing $W^{(i)}$, and Z , the optimization of $A^{(i)}$ in (6) can be formulated as follows:

$$\min_{A^{(i)T}A^{(i)}=I_a} \sum_{i=1}^{n_v} \|W^{(i)T}X^{(i)} - A^{(i)}Z\|_F^2. \quad (12)$$

Similarly to (9), we have:

$$\begin{aligned} & \min_{A^{(i)T}A^{(i)}=I_a} \sum_{i=1}^{n_v} \|W^{(i)T}X^{(i)} - A^{(i)}Z\|_F^2 \\ & \Leftrightarrow \max_{A^{(i)T}A^{(i)}=I_a} \text{Tr}(Z^T A^{(i)T} W^{(i)T} X^{(i)}) \\ & \quad + \max_{A^{(i)T}A^{(i)}=I_a} \text{Tr}(X^{(i)T} W^{(i)} A^{(i)} Z) \\ & \Leftrightarrow \max_{A^{(i)T}A^{(i)}=I_a} \text{Tr}(W^{(i)T} X^{(i)} Z^T A^{(i)T}) \\ & \Leftrightarrow \max_{A^{(i)T}A^{(i)}=I_a} \text{Tr}(A^{(i)T} P_A^{(i)}), \end{aligned} \quad (13)$$

where $P_A^{(i)} = W^{(i)T} X^{(i)} Z^T$, assuming that $P_A^{(i)}$ has the SVD form as $P_A^{(i)} = U_A^{(i)} \Sigma_A^{(i)} V_A^{(i)T}$. The solution to (12) is $A^{(i)} = U_A^{(i)} V_A^{(i)T}$.

C. Update Z

Fixing $W^{(i)}$, and $A^{(i)}$, the optimization of Z in (6) can be formulated as follows:

$$\begin{aligned} & \min_Z \sum_{i=1}^{n_v} \|W^{(i)T}X^{(i)} - A^{(i)}Z\|_F^2 \\ & \text{s.t. } Z \geq 0, \quad Z^T \mathbf{1} = \mathbf{1}, \quad \text{rank}(\tilde{L}_O) = n + a - k. \end{aligned} \quad (14)$$

Let $\sigma_i(\tilde{L}_O)$ be the i -th smallest eigenvalue of $\tilde{L}_O$. The Laplacian rank constraint is satisfied when the first k smallest eigenvalues of $\sigma_i(\tilde{L}_O)$ are equal to zero [32]. Let $P_Z = \sum_{i=1}^{n_v} A^{(i)T} (W^{(i)T} X^{(i)})$. Then, (14) transforms as follows:

$$\min_{Z \geq 0, Z^T \mathbf{1} = \mathbf{1}} \|P_Z - Z\|_F^2 + \lambda \sum_{i=1}^k \sigma_i(\tilde{L}_O). \quad (15)$$

As λ becomes sufficiently large, the first k smallest eigenvalues will converge to zero indefinitely. λ can be set either manually or automatically using heuristics. To address the problem in (15), we require an important theorem known as the Ky Fan theorem [27], expressed as follows:

$$\sum_{i=1}^k \sigma_i(\tilde{L}_O) = \min_{F^T F = I} \text{Tr}(F^T \tilde{L}_O F), \quad (16)$$

where $F = [f^1; \dots; f^{n+a}] \in \mathbb{R}^{(n+a) \times k}$ is the indicator matrix. Then, (15) can be further transformed into the following form:

$$\min_{Z \geq 0, Z^T \mathbf{1} = \mathbf{1}, F^T F = I} \|P_Z - Z\|_F^2 + \lambda \text{Tr}(F^T \tilde{L}_O F). \quad (17)$$

Therefore, (17) translates into the problem of optimizing Z and F .

1) Fix Z , and Update F : Fixing Z , optimizing F in (17) becomes:

$$\min_{F^T F = I} \text{Tr}(F^T \tilde{L}_O F). \quad (18)$$

Obviously, the optimal solution for F should consist of the k smallest eigenvectors of $\tilde{L}_O$. Considering $\tilde{L}_O = I - D_O^{-1/2} O D_O^{-1/2}$, the similarity matrix O is composed of anchor graph Z . The first n rows of F are the indicators of the raw data points, while the remaining a rows are the indicators of the anchors. Given the orthogonality of F , (18) can be solved by addressing its equivalent maximization problem as follows:

$$\max_{F_{(n)}^T F_{(n)} + F_{(a)}^T F_{(a)} = I} \text{Tr}\left(F_{(n)}^T D_{(n)}^{-1/2} Z^T D_{(a)}^{-1/2} F_{(a)}^T\right), \quad (19)$$

where,

$$F = \begin{bmatrix} F_{(n)} \\ F_{(a)} \end{bmatrix}, D_O = \begin{bmatrix} D_{(n)} & \\ & D_{(a)} \end{bmatrix}. \quad (20)$$

According to Theorem 2 [27], the solution to (19) can be obtained by performing SVD on the left and right singular values of $D_{(n)}^{-1/2} Z^T D_{(a)}^{-1/2}$. Additionally, applying k-means to F can further improve clustering accuracy.

Theorem 2: Given $T \in \mathbb{R}^{n \times m}$, $Q \in \mathbb{R}^{n \times k}$, and $P \in \mathbb{R}^{m \times k}$, the optimal solutions with respect to (21) are $Q = \sqrt{2}/2U$ and $P = \sqrt{2}/2V$, where V and U are the right and left singular vectors of T corresponding to its k largest singular values, respectively.

$$\max_{Q^T Q + P^T P = I} \text{Tr}(Q^T T P). \quad (21)$$

Algorithm 2 Optimizing Z **Input:** $W^{(i)} \in \mathbb{R}^{d_i \times m}$, $A^{(i)} \in \mathbb{R}^{m \times a}$, m , a , and λ ;**Outputs:** Z .

- 1: Initialize $W^{(i)}$, $A^{(i)}$, $F_{(n)}$, and $F_{(a)}$;
- 2: **while** Z does not have k connected components **do**
- 3: Update Z by using (24);
- 4: Update $D_{(n)}$ and $D_{(a)}$ by using (5), where $D_O(i, i) = \sum_{j=1}^{n+m} O_{ij}$;
- 5: Update F by using (19) and (20);
- 6: Adjust λ based on the Laplacian matrix $\tilde{L}_O$;
- 7: **end while**

2) Fix F , and Update Z : Given the optimal value of F , (18) can be restated as follows:

$$\begin{aligned} & \text{Tr}(F^T \tilde{L}_O F) \\ & \Leftrightarrow \sum_{i=1}^n \sum_{j=1}^a \left\| \frac{F_{(a)}(j, :)}{\sqrt{D_{(a)}(j, j)}} - \frac{F_{(n)}(i, :)}{\sqrt{D_{(n)}(i, i)}} \right\|_2^2 b_{ji}, \quad (22) \end{aligned}$$

where $F_{(a)}(j, :)$ and $F_{(n)}(i, :)$ denote the j -th and i -th rows of $F_{(a)}$ and $F_{(n)}$, respectively. $D_{(n)}(i, i)$ and $D_{(a)}(j, j)$ denote the i -th and j -th diagonal elements of $D_{(n)}$ and $D_{(a)}$, respectively. z_{ji} is the corresponding element of Z . Equation (17) can be transformed as follows:

$$\min_{Z \geq 0, Z^T \mathbf{1} = \mathbf{1}} \sum_{i=1}^n \sum_{j=1}^a (p_{ji} - z_{ji})^2 + \lambda \sum_{i=1}^n \sum_{j=1}^a t_{ij} z_{ji}, \quad (23)$$

where p_{ji} denotes the corresponding element in P_Z , $t_{ij} = \|F_{(n)}(i, :)/\sqrt{D_{(n)}(i, i)} - F_{(a)}(j, :)/\sqrt{D_{(a)}(j, j)}\|_2^2$. It is evident that the optimization of separate data is independent, and thus, we optimize $Z_{:,i}$ by columns. Consequently, the optimization in (17) can be reformulated as a solution to the following equation:

$$\min_{Z \geq 0, Z^T \mathbf{1} = \mathbf{1}} \left\| Z_{:,i} - P_{:,i} - \frac{\lambda T_{i,:}^T}{2} \right\|_2^2, \quad (24)$$

where $P_{:,i}$ denotes the i -th column element of matrix P_Z , $Z_{:,i}$ denotes the i -th column element of matrix Z , and $T_{i,:}$ denotes the row vector consisting of t_{ij} . Similar to [32], the optimal $Z_{:,i}$ can be obtained using a closed form solution. Subsequently, the optimal Z is generated by stacking the columns. We summarize the process of optimizing Z in Algorithm 2.

D. Complexity Analysis

Algorithm 3 provides a detailed procedure for the AGC model. 1) Lines 1-2 represent data preprocessing. 2) Lines 3-7 involve the training of the AGC model, conducting in-depth analysis on the raw data to obtain the corresponding Z and $A^{(i)}$. 3) Line 8 indicates the processing of the result Z . The stopping criterion is defined by conditions $(\text{obj}_{i-1} - \text{obj}_i)/\text{obj}_i \leq 10^{-4}$ and $\text{obj}_i \leq 10^{-10}$, where obj represents the objective value of the iterator.

The proposed AGC consists of three subproblems that can be efficiently solved using Algorithm 3. Due to the use of the anchor graph method, the computational complexity of AGC is relatively low. In particular, the overall time cost of

Algorithm 3 AGC**Input:** Training data $\{X^{(i)}\}_{i=1}^{n_v}$, k , m , a , and λ ;**Outputs:** Z and $A^{(i)}$.

- 1: Data preprocessing;
- 2: Initialize $W^{(i)}$, $A^{(i)}$, and Z ;
- 3: **while** non-convergence **do**
- 4: Calculate $W^{(i)}$ by using (9);
- 5: Calculate $A^{(i)}$ by using (12);
- 6: Update Z by Algorithm 2;
- 7: **end while**
- 8: After obtaining Z , a k -connected graph, we can directly apply linear graph algorithms to obtain the clustering results (or perform SVD on graph Z followed by K-means to achieve the results);

Algorithm 2 is $\mathcal{O}(na^2t + a^3t + nat)$, where t represents the number of iterations, Eq. (19) and Eq. (24) cost $\mathcal{O}(na^2 + a^3)$ and $\mathcal{O}(na)$, respectively. When optimizing $A^{(i)}$, it requires $\mathcal{O}(a^2m)$ for performing SVD and $\mathcal{O}(nam)$ for matrix multiplication. Similarly, For the $W^{(i)}$ optimization process, it requires $\mathcal{O}(m^2d_i)$ for SVD and $\mathcal{O}(nad_i)$ for matrix multiplication. Thus, the total computational complexity of Algorithm 3 is $\mathcal{O}(\sum_{i=1}^{n_v} (d_i m^2 + nad_i + ma^2 + nam) + na^2t + a^3t + nat)$. Obviously, due to $m, a, k, t \ll n$, the total computational complexity can be approximated as $\mathcal{O}(na^2t)$. In comparison, it is evident that the complexity of AGC is significantly lower than the $\mathcal{O}(n^2d)$ complexity of most graph-based methods.

V. EXPERIMENT

To minimize the impact of the experimental environment on intrusion detection performance, all comparative experiments are using computers equipped with an Intel Xeon W-2225 CPU, 64 GB of RAM, and a GTX 1060 6GB GPU running python. In our experiments, we employed the k -feature dimensionality reduction strategy, i.e. $m = k$ [36].

A. Datasets

In real-world environments, labeled IoT data is often quite scarce. Therefore, in the comparative experiments, we reduce the amount of training data and increase the amount of test data. To comprehensively and fairly validate the experimental results, we use four widely recognized network intrusion detection datasets, including UNSW-NB15,¹ CSE-CICIDS2018,² CICDDOS2019,³ and CICIOT2023.⁴ For the intrusion detection problem, we set the test set to be more than 10 times that of the training set, and the test set includes a variety of new data types not present in the training set. The dataset details are shown in Table II.

- UNSW-NB15: This dataset reflects modern network traffic scenarios and a significant number of low-traffic intrusion attacks, providing a basis for studying intrusion detection in the context of low-quality labelled data.

¹<https://research.unsw.edu.au/projects/unsw-nb15-dataset>

²<https://www.unb.ca/cic/datasets/ids-2018.html>

³<https://www.unb.ca/cic/datasets/ddos-2019.html>

⁴<https://www.unb.ca/cic/datasets/iotdataset-2023.html>

TABLE II
DATASETS

Dataset	Features	Training		Testing	
		Samples	Categories	Samples	Categories
UNSW-NB15	42	5342	10	82332	10
CSE-CICIDS2018	76	4693	9	409048	15
CICDDOS2019	76	4928	8	497458	13
CICIoT2023	39	13684	34	295885	34

- CSE-CICIDS2018: This dataset contains intrusion attacks on applications, protocols, and physical devices within real network environments, providing valuable data for network security research.
- CICDDOS2019: The dataset includes common distributed denial of service attacks observed in contemporary networks and is suitable for application in various IoT scenarios.
- CICIoT2023: The dataset consists of various types of intrusion attacks conducted within an IoT topology composed of multiple IoT devices, where all attacks are executed by malicious IoT devices targeting other devices.

B. Experiment Setup

We evaluated the effectiveness of AGC in intrusion detection, focusing on the following key aspects: 1) comparing AGC with state-of-the-art algorithms, 2) validating the performance of our approach in different scenarios, 3) assessing the computational overhead of AGC, and 4) evaluating the sensitivity of AGC. To this end, we designed eight experimental scenarios, detailed as follows.

For the comparative experiments with low-quality labeled data, this paper uses the following metrics: accuracy (ACC), and false alarm rate (FAR) [15]. To compare and evaluate the performance of AGC on the mentioned datasets, we chose several representative detection methods, including both traditional ML and DL approaches. These comparison methods include three traditional ML methods, i.e., an intrusion detection method based on the K-nearest neighbors algorithm and the arithmetic optimization algorithm (KNN+AO) [37], a detection algorithm that combines the crow search algorithm, particle swarm optimization, and random forest (CSA+PSO+RF) [38], and an intrusion detection technique based on regression models and linear support vector machine (RM+LSVM) [39]; four DL methods, i.e., an IoT intrusion detection method based on fuzzy rough sets and conditional generative adversarial networks (RS+CGAN) [15], an adaptive intrusion detection system utilizing advanced DL techniques with stacked ensemble integration (AdaptIDS) [40], a graph neural network approach utilizing traffic interaction graphs (GNN+TIG) [41], and a robust graph neural network detection method under adversarial attacks (RGNN) [42]; and three combined ML and DL methods, i.e., a detection model combining stacked contractive auto-encoder and SVM for cloud IDS (SCAE+SVM) [43], a multi-task IoT intrusion detection model based on autoencoders and SVM (MAE+SVM) [44],

and a classification technique combining graph embedding technology and random forest (Graph2vec+RF) [45]. All the compared methods represent the latest intrusion detection approaches. For different tasks, we maintained the overall architecture unchanged and only modified the outputs of the aforementioned comparison methods for validation.

Experiment 1: Comparison with the latest intrusion detection methods under conditions of complete labels and data loss. In this experiment, we aimed to examine whether our approach is competitive with existing methods under fully labeled conditions. Using the aforementioned evaluation datasets, we trained all intrusion detection models with 10% of the original labels and compared their performance. Subsequently, we incrementally increased the amount of labeled data to observe the benefits of utilizing correctly labeled data. In this experiment, we conducted both binary classification and multi-class classification comparisons.

Experiment 2: Comparison with the latest intrusion detection methods under missing label conditions. Specifically, we constructed a training dataset to simulate scenarios where some class labels are missing. We randomly selected k_c classes and removed all labels from the training samples of these classes. For the remaining classes, we retained only 10% of the labeled samples for each class. In this experiment, we varied k_c to examine its impact on system performance.

Experiment 3: Comparison with the latest intrusion detection methods under coarse-grained label conditions. Assume the dataset initially contains k classes. We randomly select k_g classes and merge their training samples into a single joint label. For the remaining classes, we retain only 10% of the labeled training samples for each class. With this setup, the training set contains only $k - k_g + 1$ classes in total, each with 10% of its data labeled. We varied k_g to examine its impact on system performance. In this experiment, we focused on scenarios where all other classes are merged into a single joint label. Scenarios involving multiple joint labels yield conclusions consistent with the single joint label case, so the results for multiple joint labels are omitted here.

In the existing IoT environments, collecting tagged data is challenging, time-consuming, and sometimes impossible. Therefore, exploring the unsupervised clustering capabilities of algorithms can significantly advance the development of IDS. Moreover, unsupervised learning methods often outperform approaches such as semi-supervised learning, which rely on labeled data [25]. AGC is built based on unsupervised techniques and can cluster unlabeled intrusion data. For experiments on unlabeled data, we use the following three metrics: ACC, purity (PUR), and normalized mutual information (NMI) [25]. To compare and evaluate the performance of AGC for unsupervised clustering on the aforementioned datasets, we selected several representative unsupervised methods for comparison, including mutual information-driven multi-view clustering (MIMC) [33], scalable multi-view subspace clustering (SMVSC) [26], Auto-weighted Multiple Graph Learning (AMGL) [46], Fast Parameter-Free Multi-View Subspace Clustering (FPMVSC) [32], and Adaptive Graph Learning and Matrix Factorization (AGLMF) [47].

TABLE III
BINARY CLASSIFICATION RESULTS UNDER FULLY LABELED DATA

Dataset	Rate	Metric	KNN+AO	CSA+PSO+RF	RM+LSVM	RS+CGAN	AdaptIDS	GNN+TIG	RGNN	SCAE+SVM	MAE+SVM	Graph2vec+RF	AGC
UNSW-NB15	100%	ACC	84.26%	84.10%	83.83%	87.07%	91.00%	87.79%	86.88%	83.78%	83.96%	85.63%	84.93%
		FAR	27.41%	34.07%	32.16%	18.23%	26.75%	23.14%	25.67%	27.06%	25.54%	25.51%	13.11%
	50%	ACC	84.50%	84.76%	80.87%	84.82%	78.11%	79.34%	84.88%	82.43%	83.23%	84.73%	84.93%
		FAR	26.64%	32.63%	42.35%	26.11%	87.71%	27.81%	24.27%	25.21%	24.91%	24.70%	13.11%
	10%	ACC	81.95%	80.80%	76.64%	79.96%	55.25%	73.25%	81.80%	75.51%	75.50%	81.60%	82.07%
		FAR	31.37%	41.41%	51.86%	43.05%	100.00%	33.35%	31.34%	30.68%	30.42%	31.22%	18.22%
CSE-CIC IDS2018	100%	ACC	88.62%	89.10%	88.12%	88.23%	99.78%	91.73%	90.36%	89.57%	89.12%	88.80%	90.17%
		FAR	27.51%	21.62%	16.15%	49.17%	10.00%	17.54%	13.18%	29.03%	29.71%	18.36%	27.39%
	50%	ACC	88.79%	89.98%	87.90%	87.31%	88.14%	85.64%	84.26%	87.67%	86.57%	87.66%	90.17%
		FAR	29.92%	23.13%	27.14%	37.92%	18.21%	20.45%	25.43%	32.16%	33.43%	20.71%	27.39%
	10%	ACC	87.43%	87.07%	86.22%	83.49%	66.48%	76.75%	78.54%	85.17%	79.00%	88.27%	90.17%
		FAR	36.88%	30.15%	33.82%	51.29%	32.17%	34.13%	35.63%	39.40%	54.19%	29.41%	27.39%
CICDDOS 2019	100%	ACC	91.67%	82.82%	90.04%	93.87%	95.10%	94.74%	93.66%	93.76%	92.12%	93.23%	92.99%
		FAR	1.79%	3.36%	1.45%	26.53%	0.41%	1.73%	1.88%	3.19%	3.45%	1.51%	3.19%
	50%	ACC	90.74%	81.56%	87.13%	90.31%	79.61%	90.79%	90.19%	90.40%	88.67%	90.37%	92.99%
		FAR	1.68%	3.09%	1.03%	21.90%	0.18%	2.37%	3.27%	12.37%	11.25%	7.20%	3.19%
	10%	ACC	90.47%	80.64%	85.13%	91.05%	63.84%	78.61%	78.54%	87.31%	86.47%	87.09%	92.99%
		FAR	6.19%	4.12%	3.54%	44.59%	33.33%	14.05%	15.34%	13.67%	12.52%	9.84%	3.19%
CICIoT 2023	100%	ACC	95.85%	95.97%	95.45%	99.35%	98.19%	99.17%	96.39%	96.10%	95.67%	97.08%	97.46%
		FAR	62.51%	68.34%	78.31%	7.98%	1.81%	2.57%	4.25%	58.29%	41.77%	55.56%	10.12%
	50%	ACC	95.69%	95.87%	94.65%	98.67%	85.81%	97.17%	93.51%	95.94%	89.14%	97.08%	97.46%
		FAR	64.95%	70.24%	93.56%	22.44%	14.19%	23.71%	31.59%	60.29%	43.57%	61.61%	10.12%
	10%	ACC	94.80%	95.46%	94.63%	95.53%	59.50%	95.56%	91.64%	93.75%	77.76%	92.73%	96.46%
		FAR	60.54%	76.99%	93.63%	52.01%	40.50%	48.67%	49.46%	60.99%	51.47%	61.57%	13.57%

TABLE IV
MULTI-CLASS CLASSIFICATION RESULTS UNDER FULLY LABELED DATA

Dataset	Rate	Metric	KNN+AO	CSA+PSO+RF	RM+LSVM	RS+CGAN	AdaptIDS	GNN+TIG	RGNN	SCAE+SVM	MAE+SVM	Graph2vec+RF	AGC
UNSW-NB15	100%	ACC	76.59%	81.58%	73.28%	81.99%	87.69%	80.45%	80.50%	79.05%	72.34%	79.24%	81.73%
	10%	ACC	71.19%	65.30%	56.32%	29.91%	31.89%	29.18%	25.90%	65.11%	63.03%	66.68%	78.12%
CSE-CIC IDS2018	100%	ACC	78.67%	79.33%	72.81%	69.81%	68.43%	77.94%	78.02%	78.30%	62.73%	78.61%	83.03%
	10%	ACC	76.56%	76.38%	72.82%	32.42%	36.63%	37.66%	38.18%	73.44%	47.72%	74.86%	83.03%
CICDDOS 2019	100%	ACC	48.63%	41.80%	37.11%	54.60%	43.81%	49.10%	50.09%	50.61%	48.71%	51.36%	85.47%
	10%	ACC	46.75%	41.44%	31.42%	29.19%	25.15%	29.09%	36.52%	47.30%	38.76%	47.05%	85.47%
CICIoT 2023	100%	ACC	68.09%	73.45%	72.06%	90.93%	88.86%	87.28%	82.76%	72.23%	70.72%	72.37%	86.67%
	10%	ACC	66.08%	69.96%	65.35%	49.64%	40.66%	57.98%	56.83%	64.54%	58.83%	63.61%	82.29%

Experiment 4: Comparison with the latest unsupervised methods under the known number of types k . In this experiment, we examine the performance of AGC in unsupervised classification. Given 10% of the aforementioned evaluation datasets, we first remove all labels and compare AGC with existing unsupervised algorithms, where the number of data types k is known.

Experiment 5: Comparison with the latest unsupervised methods when the type k is unknown. Given 10% of the aforementioned evaluation datasets, we first remove all labels and compare AGC with existing unsupervised algorithms, where the number of data types k is unknown. We change the k value to check its impact on all methods.

Experiment 6: Running time comparison. To evaluate the computational cost of AGC, we also record the overall training time and testing time for each method.

Experiment 7: Sensitivity analysis. We assess the sensitivity of the AGC model to changes in input variables or hyperparameters.

Experiment 8: Ablation experiments. We verify the role of each module in the AGC model.

C. Experiment Results

1) Comparison With the Latest Intrusion Detection Methods Under Conditions of Complete Labels and Data Loss:

Table III shows the binary classification performance comparison between our AGC method and other methods, while Table IV provides the performance comparison for multi-class classification. From these results, we can conclude that AGC outperforms other methods as the amount of labeled data decreases. Additionally, with reduced labeled data, DL methods tend to experience a significant increase in FAR or a substantial decline in ACC, making them unsuitable for scenarios with label loss. Traditional ML methods and hybrid approaches that combine them demonstrate stronger adaptability in handling label loss scenarios. However, as the amount of label loss increases, their detection ACC falls below that of our proposed method.

The results from the two tables indicate that the method proposed in this paper also experiences a decline in accuracy when data labeling decreases. This is because the reduction in labels causes the boundaries between different data types to become blurred, which further leads to changes in the

TABLE V
COMPARISON RESULTS UNDER DIFFERENT INDEXES

Dataset	Index	Metric	KNN+AO	CSA+PSO+RF	RM+LSVM	RS+CGAN	AdaptIDS	GNN+TIG	RGNN	SCAE+SVM	MAE+SVM	Graph2vec+RF	AGC
UNSW-NB15	$k_c = 2$	ACC	62.57%	60.51%	49.61%	29.51%	30.14%	27.80%	24.61%	54.02%	53.21%	56.54%	75.40%
	$k_c = 5$	ACC	48.29%	46.51%	47.73%	28.29%	27.86%	26.85%	22.56%	41.16%	39.71%	49.25%	71.27%
	$k_g = 2$	ACC	66.43%	61.49%	48.98%	27.33%	30.93%	27.72%	24.66%	52.74%	54.40%	61.37%	72.65%
	$k_g = 5$	ACC	47.24%	43.73%	39.08%	22.87%	21.76%	24.85%	22.91%	44.88%	32.87%	47.48%	70.82%
CSE-CICIDS2018	$k_c = 2$	ACC	61.32%	60.68%	60.02%	29.83%	36.50%	37.26%	36.85%	61.30%	36.47%	62.49%	83.03%
	$k_c = 5$	ACC	46.08%	45.94%	46.53%	26.68%	32.21%	32.29%	28.85%	45.79%	28.14%	45.92%	75.69%
	$k_g = 2$	ACC	62.18%	61.15%	60.84%	31.59%	34.69%	36.18%	34.21%	62.25%	31.26%	60.46%	82.36%
	$k_g = 5$	ACC	37.74%	39.44%	41.20%	28.94%	27.73%	28.16%	25.75%	37.05%	25.30%	36.49%	71.77%
CICDDOS2019	$k_c = 2$	ACC	37.12%	33.87%	28.42%	28.35%	22.91%	28.35%	34.81%	39.90%	36.64%	40.61%	81.11%
	$k_c = 5$	ACC	27.39%	28.45%	23.88%	21.24%	20.37%	22.31%	27.79%	28.35%	26.03%	28.30%	75.68%
	$k_g = 2$	ACC	42.28%	35.32%	23.25%	28.06%	24.63%	27.56%	35.42%	42.01%	35.52%	42.78%	83.05%
	$k_g = 5$	ACC	27.38%	21.38%	21.28%	21.30%	20.29%	22.35%	27.67%	27.51%	23.45%	27.63%	72.95%
CICIoT2023	$k_c = 2$	ACC	60.58%	67.78%	60.86%	47.64%	38.74%	55.64%	54.29%	60.22%	50.92%	61.85%	80.48%
	$k_c = 5$	ACC	58.34%	61.38%	58.22%	46.65%	33.29%	50.35%	51.17%	56.60%	43.98%	58.29%	77.51%
	$k_g = 2$	ACC	65.81%	68.14%	64.83%	49.30%	37.00%	56.75%	56.32%	60.70%	55.16%	62.67%	82.29%
	$k_g = 5$	ACC	64.12%	67.48%	62.29%	47.89%	35.08%	54.30%	54.29%	58.38%	44.13%	61.20%	81.68%

decision thresholds of the anchor points. This work is the first attempt to apply anchor graph learning to the field of intrusion detection, and the experimental results demonstrate that AGC can be flexibly applied to various attack scenarios. As shown in the two tables, although AGC achieves high detection ACC and relatively low FAR on most datasets, it exhibits a higher FAR on the UNSW-NB15 and CSE-CICIDS2018 datasets. This phenomenon is partly attributed to the class imbalance in the dataset and the high similarity of features among some malicious behaviors. Additionally, it is influenced by the fact that the test set contains more types of data than the training set. Under such circumstances, traditional classification methods are prone to misclassifying certain normal behaviors as attacks. DL methods can address the above issues through fitting; however, as the availability of labeled data decreases, their fitting accuracy declines, leading to an increase in FAR. While the AGC method exhibits a relatively higher FAR, this increase contributes to AGC achieving higher ACC compared to traditional ML methods. Furthermore, AGC is a lightweight model. In the subsequent experiments, we compared the runtime of various models, and the results show that AGC meets the lightweight requirements of IoT devices.

2) *Comparison With the Latest Intrusion Detection Methods Under Missing Label Conditions:* With 10% of the training data, we randomly remove the labels of k_c classes, and the comparison results are shown in Table V. From the conclusions in Table IV, it can be observed that due to limited labels, existing DL methods do not outperform traditional ML methods. Furthermore, as the number of missing labels increases, the performance of comparison methods begins to decline. For instance, when k_c increases from 2 to 5, the ACC of some comparison methods drops by approximately 15%. Since current intrusion detection methods heavily rely on fully labeled data and perform better with more data, they are unable to make accurate predictions for test samples belonging to unlabeled classes in the absence of such information. Additionally, due to the issue of class imbalance, the selection of missing classes can also impact the accuracy of intrusion detection methods. In contrast, AGC demonstrates higher

detection accuracy in the comparison experiments. In these experiments, missing classes affect the adjustment of threshold boundaries corresponding to anchor points in AGC, thereby influencing the final results. Combining the observations from Tables III to V, we can conclude that when fewer labels or missing class labels are provided, AGC exhibits significant advantages.

3) *Comparison With the Latest Intrusion Detection Methods Under Coarse-Grained Label Conditions:* Table V also presents the comparison results under the coarse-grained label settings. As k_g increases, the performance of the comparison methods begins to degrade, highlighting the inability of existing intrusion detection methods to handle coarse-grained labels effectively. In contrast, the proposed method demonstrates higher detection accuracy when dealing with coarse-grained labels. The table also reveals that when data classes are imbalanced, setting the majority class types as coarse-grained leads to a significant drop in ACC, as observed in the CSE-CICIDS2018 dataset results. Conversely, when minority class types are set as coarse-grained labels, the impact is less pronounced, as seen in the results from the CICIoT2023 dataset. For AGC, coarse-grained labels may mislead the estimation of the true number of classes and disrupt the effectiveness of clustering integration, particularly in cases of highly imbalanced classes.

From the first three experiments, it can be concluded that the proposed method demonstrates superior detection capabilities in handling low-quality data scenarios, outperforming existing intrusion detection methods. This is because our approach leverages clustering during the learning process to reveal the intrinsic distribution of samples and identify representative anchor points. The class represented by each anchor point is jointly determined by multiple labeled data points and clustering results, making it less susceptible to the impact of low-quality labeled data.

4) *Comparison With the Latest Unsupervised Methods Under the Known Number of Types k :* As shown in Table VI, we compare the proposed AGC with five clustering algorithms. Compared to traditional static anchor graph generation methods like AGLMF, our method learns anchor graph dynamically

TABLE VI
RESULTS OF UNSUPERVISED CLUSTERING (K IS KNOWN)

Dataset	k	Metric	MIMC	SMVSC	AMGL	FPMVS	AGLMF	AGC
UNSW-NB15	10	ACC	31.00%	41.00%	33.60%	36.00%	35.80%	42.00%
		NMI	26.30%	33.34%	28.11%	29.63%	34.87%	36.82%
		PUR	53.60%	42.40%	36.60%	38.20%	40.40%	44.00%
CSE-CICIDS2018	9	ACC	54.17%	63.33%	38.61%	60.83%	48.06%	74.72%
		NMI	59.51%	60.58%	51.28%	61.88%	60.65%	81.47%
		PUR	67.22%	63.33%	45.28%	61.94%	55.00%	76.39%
CICDDOS2019	8	ACC	38.50%	56.50%	54.00%	63.25%	49.00%	70.75%
		NMI	42.07%	51.60%	60.56%	59.23%	48.42%	61.77%
		PUR	77.25%	57.50%	57.00%	63.50%	50.25%	70.75%
CICIoT2023	34	ACC	31.57%	39.31%	35.29%	43.04%	37.94%	47.06%
		NMI	60.04%	55.14%	60.71%	58.08%	61.75%	62.39%
		PUR	65.00%	40.69%	41.96%	44.02%	44.22%	48.14%

TABLE VII
RESULTS OF UNSUPERVISED CLUSTERING (K IS RANDOMLY SELECTED)

Dataset	k	Metric	MIMC	SMVSC	AMGL	FPMVS	AGLMF	AGC
UNSW-NB15	2	ACC	17.20%	17.20%	13.20%	18.60%	18.20%	41.40%
		NMI	7.94%	7.88%	6.46%	10.12%	10.90%	31.83%
		PUR	70.00%	17.20%	13.20%	18.60%	18.20%	42.60%
	5	ACC	22.80%	33.20%	22.00%	28.60%	28.00%	41.40%
		NMI	18.31%	27.51%	21.55%	22.27%	26.75%	31.83%
		PUR	62.20%	33.20%	24.40%	28.60%	30.40%	42.60%
CSE-CICIDS2018	2	ACC	22.22%	22.22%	22.22%	22.22%	16.94%	74.72%
		NMI	27.85%	31.70%	27.40%	31.55%	12.03%	81.47%
		PUR	83.06%	22.22%	22.22%	22.22%	16.94%	76.39%
	5	ACC	39.44%	48.33%	26.39%	46.11%	38.33%	74.72%
		NMI	48.40%	50.04%	44.46%	49.07%	46.12%	81.47%
		PUR	71.67%	48.33%	33.06%	46.11%	38.33%	76.39%
CICDDOS2019	2	ACC	12.75%	25.00%	24.75%	25.00%	21.75%	70.75%
		NMI	0.50%	30.31%	24.73%	37.99%	16.90%	61.77%
		PUR	99.75%	25.00%	24.75%	25.00%	21.75%	70.75%
	5	ACC	31.75%	48.75%	46.25%	49.25%	41.50%	70.75%
		NMI	36.89%	52.68%	50.61%	46.98%	42.63%	61.77%
		PUR	88.00%	48.75%	46.25%	49.25%	41.50%	70.75%
CICIoT2023	2	ACC	4.80%	5.88%	5.88%	5.88%	5.59%	43.53%
		NMI	11.63%	17.03%	11.38%	17.64%	6.18%	60.80%
		PUR	82.06%	5.88%	5.88%	5.88%	5.59%	45.39%
	5	ACC	9.22%	14.71%	12.16%	14.41%	11.57%	43.53%
		NMI	26.14%	41.98%	21.10%	41.40%	21.94%	60.80%
		PUR	75.49%	14.71%	12.16%	14.41%	11.57%	45.39%

while filtering out data noise. In addition, by imposing constraints on the anchor graph, our AGC method can directly generate clustering results. In summary, the dynamic anchor learning method effectively improves anchor quality and enhances clustering accuracy. As shown in the table, when the data combination is complex, the accuracy of AGC is lower, as seen in the UNSW-NB15 and CICIoT2023 datasets. Conversely, the clearer the data structure, the higher the detection accuracy of AGC, as demonstrated by the CSE-CICIDS2018 and CICDDOS2019 datasets.

5) *Comparison With the Latest Unsupervised Methods Under the Unknown Number of Types k* : The superior performance of existing unsupervised methods heavily relies on an accurate number of types, which is not always available in real-world scenarios. Therefore, Table VII validates the comparative results between existing methods and AGC under unsupervised conditions when the number of data types k is incorrectly provided.

By comparing Table VI and Table VII, it can be observed that when k is inaccurate, the detection performance of all algorithms declines. Existing algorithms, guided by an incorrect predefined number of clusters, experience a significant performance drop, rendering them incapable of handling scenarios with unknown data types. Unlike existing methods, the approach proposed in this paper enhances the analysis of the underlying structure of the data, enabling the estimation of an approximate k value to further guide the training of the AGC model. As shown in Table VII, when the data structure is well-defined, the proposed method maintains validation results comparable to those achieved with a known k value, as demonstrated by the clustering results on the CSE-CICIDS2018 and CICDDOS2019 datasets. However, when the data types are overly complex, such as in the CICIoT2023 dataset, the proposed method exhibits performance degradation and struggles to accurately capture the k value. Nevertheless, compared to existing methods, the proposed approach is better equipped to handle scenarios with unknown data types.

6) *Running Time Comparison*: The detection runtime of all supervised intrusion detection algorithms are shown in Table VIII, where AGC (k) represents the runtime of our proposed method under the condition that the original data types are determined. As shown in the table, due to the need to identify the true number of original data types, the training time of AGC is longer than that of traditional methods (e.g., CSA+PSO+RF), and its testing time is also slightly higher. However, when the determination of the number of types is not required, AGC demonstrates similar efficiency to traditional ML methods. Compared to the latest DL methods, AGC requires less time for both training and testing. The primary time consumption of AGC lies in determining the number of original data types. Analyzing Table VIII, we can conclude that, compared to existing methods, AGC has a relatively fast detection time while offering advantages in terms of detection performance. The time complexity of AGC is $\mathcal{O}(na^2t)$, and it does not have a complex model structure, which results in considerable operational efficiency. Compared to most DL methods, AGC has significant advantages in both efficiency and accuracy.

7) *Sensitivity Analysis*: In the proposed AGC method, there are two parameters, a , and λ need to be accurately set. Fig. 4 illustrates the parameter sensitivity analysis of our method. Using the widely adopted grid search scheme, we adjust the range of a within $[k, 2k, 3k, 4k, 5k, 10k, 30k]$, and λ set in the range of $[10^{-3}, 10^{-2}, 1, 100, 1000]$. As shown in Fig. 4, AGC is less sensitive to λ . In addition, these figures show that the sensitivity of the parameters is related to the number of anchors a . The following conclusions can be drawn effortlessly: (1) in general, a small number of anchors has difficulty capturing the true distribution of data in large datasets; (2) too many anchors can lead to a slight decrease in clustering performance, as they introduce a more complex and confusing graph structure.

8) *Ablation Experiments*: As shown in (6), AGC projects the raw points into an embedded space and learns a unified anchor graph to approximate the clusters in the network data. Therefore, it is necessary to perform an ablation analysis to investigate the contributions of dynamic anchor graph

TABLE VIII
MODEL TRAINING AND TESTING TIME

Dataset	Metric	KNN+AO	CSA+PSO+RF	RM+LSVM	RS+CGAN	AdaptIDS	GNN+TIG	RGNN	SCAE+SVM	MAE+SVM	Graph2vec+RF	AGC	AGC (k)
UNSW-NB15	Training (s)	1.53	1.78	14.47	602.44	475.79	377.86	357.12	212.06	49.93	67.54	37.77	3.61
	Testing (s)	3.20	3.45	6.37	149.82	12.29	11.09	10.11	11.33	10.66	9.43	3.67	3.67
CSE-CIC-IDS2018	Training (s)	1.05	1.10	8.15	395.07	422.29	353.89	325.79	196.61	42.28	55.06	31.57	2.72
	Testing (s)	3.01	3.88	8.45	134.82	11.74	10.09	11.2	10.87	9.21	6.32	3.10	3.10
CICDDOS 2019	Training (s)	0.98	1.46	6.27	394.96	381.86	346.15	320.62	195.34	41.75	55.96	35.73	3.89
	Testing (s)	3.00	3.27	3.40	179.58	15.53	26.15	22.10	8.73	6.77	7.76	3.56	3.56
CICIoT 2023	Training (s)	0.88	1.46	6.68	416.39	517.65	366.60	336.33	209.26	42.79	61.76	38.29	5.71
	Testing (s)	3.93	3.27	5.74	169.58	17.53	18.15	19.15	9.46	10.11	9.60	3.60	3.60

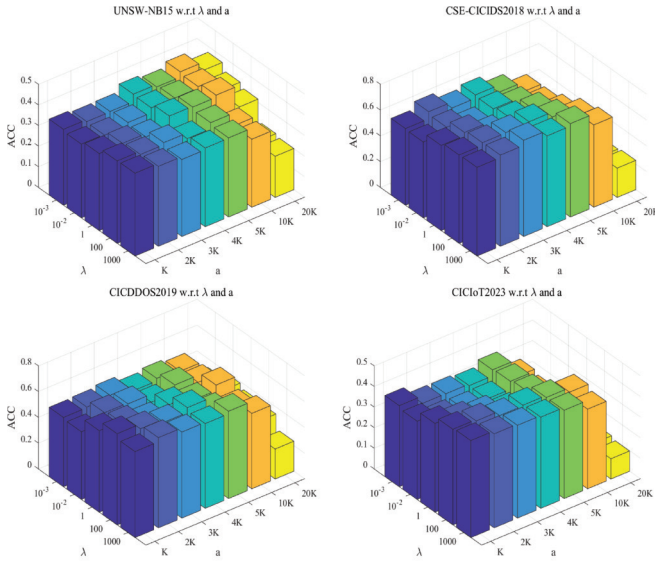


Fig. 4. Parameter sensitivity.

clustering and embedding learning. To verify the effectiveness of AGC, the different variations are described as follows.

- **S1:** Full AGC model.
- **S2:** To validate the effectiveness of embedding learning, we removed W .
- **S3:** To validate the effectiveness of anchor learning, we removed A .
- **S4:** To validate the effectiveness of dynamic anchor learning, we utilized the method in [48] to pre-select the anchors and replace A with static anchor points.
- **S5:** To validate the effectiveness of unified joint learning, we removed the rank constraint and used K-means to cluster the anchor graph.
- **S6:** To validate the importance of determining the number of clusters, we removed the cluster determination step and randomly selected $k = 2$ for verification.

Table IX shows the ablation experiment in the unsupervised scenario. From the results of the four experimental datasets in the two tables, it can be concluded that AGC outperforms other ablation methods. Compared to **S2**, it can be seen that embedding learning reduces the uncertainty in the dataset and improves the recognition accuracy. Compared to **S3** and **S4**, it can be seen that the dynamic learning of anchor graphs helps to improve the overall performance. Compared to **S5**, unified learning achieves better clustering performance than

TABLE IX
ABLATION EXPERIMENTS FOR AGC

Dataset	Metric	S1	S2	S3	S4	S5	S6
UNSW-NB15	ACC	42.00%	30.80%	23.20%	41.00%	41.00%	19.20%
	NMI	36.82%	26.88%	21.77%	35.04%	33.33%	13.47%
	PUR	44.00%	49.00%	58.60%	41.80%	42.4%	65.60%
CSE-CIC-IDS2018	ACC	74.72%	25.83%	45.56%	62.78%	70.83%	25.56%
	NMI	81.47%	37.59%	47.72%	64.40%	71.65%	32.37%
	PUR	76.39%	83.61%	71.94%	63.33%	70.83%	92.50%
CICDDOS 2019	ACC	70.75%	39.50%	31.75%	66.75%	68.25%	27.25%
	NMI	70.75%	38.60%	37.78%	62.41%	57.87%	28.83%
	PUR	70.75%	68.50%	85.75%	68.50%	68.25%	80.00%
CICIoT 2023	ACC	47.06%	28.14%	3.82%	43.04%	46.37%	5.88%
	NMI	62.39%	54.70%	8.27%	58.08%	61.84%	18.97%
	PUR	48.14%	69.31%	95.88%	44.02%	47.25%	5.88%

multi-step clustering, as the former dynamically optimizes various parameters effectively. The comparison in **S6** demonstrates that an accurate number of clusters can improve clustering accuracy and contribute to intrusion detection. Furthermore, AGC outperforms the ablated versions on all datasets, confirming the importance of embedding learning and the use of high-quality anchor graph. Although embedding learning provides only a small improvement in precision, it lays the foundation for multi-view and multi-task learning.

VI. CONCLUSION

In this work, we propose an intrusion detection method for low-quality labels to enhance the resilience of IDSs in IoT networks. The proposed AGC method first reduces the dimensions of the raw data through embedding learning. Then, AGC unifies anchor graph construction, graph learning, and graph clustering into a single framework by dynamically learning the anchor matrix. Finally, we achieve intrusion detection based on the constructed anchor graph. To validate the method proposed in this work, we compared AGC with current leading detection algorithms. The results indicate that, compared to existing methods, the proposed approach achieves higher detection accuracy in low-quality scenarios. Furthermore, the algorithm designed in this paper has low complexity, making it well-suited for IoT environments.

Considering that imbalanced data in IoT environments can lead to a decline in detection performance, as well as the need to enhance the proposed method's capability to handle large-scale data, we plan to further improve the anchor graph generation process in future work. This improvement aims to better address highly imbalanced and complex datasets.

REFERENCES

- [1] A. Jamalipour and S. Murali, "A taxonomy of machine-learning-based intrusion detection systems for the Internet of Things: A survey," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9444–9466, Jun. 2022.
- [2] K. Sood, M. R. Nosouhi, D. D. N. Nguyen, F. Jiang, M. Chowdhury, and R. Doss, "Intrusion detection scheme with dimensionality reduction in next generation networks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 965–979, 2023.
- [3] H. Ding, Y. Sun, N. Huang, Z. Shen, and X. Cui, "TMG-GAN: Generative adversarial networks-based imbalanced learning for network intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 1156–1167, 2024.
- [4] R. Kalakoti, H. Bahsi, and S. Nömm, "Improving IoT security with explainable AI: Quantitative evaluation of explainability for IoT botnet detection," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 18237–18254, May 2024.
- [5] A. Oseni et al., "An explainable deep learning framework for resilient intrusion detection in IoT-enabled transportation networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 1000–1014, Jan. 2023.
- [6] J. Cai, J. Fan, W. Guo, S. Wang, Y. Zhang, and Z. Zhang, "Efficient deep embedded subspace clustering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 1–10.
- [7] L. Yao, W. Niu, Q. Yuan, B. Li, Y. Zhang, and X. Zhang, "A robust malicious traffic detection framework with low-quality labeled data," in *Proc. IEEE Int. Conf. Commun.*, Jun. 2024, pp. 2719–2724.
- [8] Y. Xing, D. Pandey, F. Wu, M. Fronda, and P. Bhattacharya, "Training with low-label-quality data: Rank pruning and multi-review," in *Proc. Mach. Learn. Res.*, 2023, pp. 1–11.
- [9] M. Bakaev and V. Khvorostov, "Quality of labeled data in machine learning: Common sense and the controversial effect for user behavior models," *Eng. Proc.*, vol. 33, no. 1, p. 3, 2023.
- [10] A. Tellache, A. Mokhtari, A. A. Korba, and Y. Ghamri-Doudane, "Multi-agent reinforcement learning-based network intrusion detection system," in *Proc. IEEE Netw. Operations Manage. Symp.*, May 2024, pp. 1–9.
- [11] X. Li, Y. Sun, Q. Sun, Z. Ren, and Y. Sun, "Cross-view graph matching guided anchor alignment for incomplete multi-view clustering," *Inf. Fusion*, vol. 100, Dec. 2023, Art. no. 101941.
- [12] D. Shi, L. Zhu, J. Li, Z. Cheng, and Z. Zhang, "Flexible multiview spectral clustering with self-adaptation," *IEEE Trans. Cybern.*, vol. 53, no. 4, pp. 2586–2599, Apr. 2023.
- [13] J. Lai, T. Wang, C. Chen, and Z. Zheng, "Information-aware multi-view outlier detection," *ACM Trans. Knowl. Discovery Data*, vol. 18, no. 4, pp. 1–16, May 2024.
- [14] W. Liang, K.-C. Li, J. Long, X. Kui, and A. Y. Zomaya, "An industrial network intrusion detection algorithm based on multifeature data clustering optimization model," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 2063–2071, Mar. 2020.
- [15] Y. Wu, L. Nie, S. Wang, Z. Ning, and S. Li, "Intelligent intrusion detection for Internet of Things security: A deep convolutional generative adversarial network-enabled approach," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3094–3106, Feb. 2023.
- [16] E. Benkhelifa, T. Welsh, and W. Hamouda, "A critical review of practices and challenges in intrusion detection systems for IoT: Toward universal and resilient systems," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 3496–3509, 4th Quart., 2018.
- [17] S. Latif et al., "Intrusion detection framework for the Internet of Things using a dense random neural network," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6435–6444, Sep. 2022.
- [18] M. Malik, M. Dutta, and J. Granjal, "IoT-sentry: A cross-layer-based intrusion detection system in standardized Internet of Things," *IEEE Sensors J.*, vol. 21, no. 24, pp. 28066–28076, Dec. 2021.
- [19] A. A. Korba, A. Boulouache, and Y. Ghamri-Doudane, "Zero-X: A blockchain-enabled open-set federated learning framework for zero-day attack detection in IoV," *IEEE Trans. Veh. Technol.*, vol. 73, no. 9, pp. 12399–12414, Sep. 2024.
- [20] A. Diaf, A. A. Korba, N. Elislem Karabadi, and Y. Ghamri-Doudane, "Beyond detection: Leveraging large language models for cyber attack prediction in IoT networks," in *Proc. 20th Int. Conf. Distrib. Comput. Smart Syst. Internet Things (DCOSS-IoT)*, Apr. 2024, pp. 117–123.
- [21] N. Hu, Z. Tian, H. Lu, X. Du, and M. Guizani, "A multiple-kernel clustering based intrusion detection scheme for 5G and IoT networks," *Int. J. Mach. Learn. Cybern.*, vol. 12, pp. 1–16, Jan. 2021.
- [22] N. Iam-On, "Clustering data with the presence of attribute noise: A study of noise completely at random and ensemble of multiple k-means clusterings," *Int. J. Mach. Learn. Cybern.*, vol. 11, no. 3, pp. 491–509, Mar. 2020.
- [23] L. Cerdá-Alabern, G. Iuhász, and G. Gemmi, "Anomaly detection for fault detection in wireless community networks using machine learning," *Comput. Commun.*, vol. 202, pp. 191–203, Mar. 2023.
- [24] J. Cai, Y. Zhang, Z. Lu, W. Guo, and S.-K. Ng, "Towards effective federated graph anomaly detection via self-boosted knowledge distillation," in *Proc. ACM Int. Conf. Multimedia*, Oct. 2024, pp. 5537–5546.
- [25] Z. Lu, F. Nie, R. Wang, and X. Li, "A differentiable perspective for multi-view spectral clustering with flexible extension," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 6, pp. 7087–7098, Jun. 2023.
- [26] C. Zhang, X. Jia, Z. Li, C. Chen, and H. Li, "Learning cluster-wise anchors for multi-view clustering," in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2024, vol. 38, no. 15, pp. 16696–16704.
- [27] X. Li, H. Zhang, R. Wang, and F. Nie, "Multiview clustering: A scalable and parameter-free bipartite graph fusion method," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 1, pp. 330–344, Jan. 2022.
- [28] Z. Kang, Z. Lin, X. Zhu, and W. Xu, "Structured graph learning for scalable subspace clustering: From single view to multiview," *IEEE Trans. Cybern.*, vol. 52, no. 9, pp. 8976–8986, Sep. 2022.
- [29] W. Xia, Q. Gao, Q. Wang, X. Gao, C. Ding, and D. Tao, "Tensorized bipartite graph learning for multi-view clustering," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 5187–5202, Apr. 2023.
- [30] A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, "A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1851–1877, 2nd Quart., 2019.
- [31] J. Liang, W. Guo, T. Luo, V. Honavar, G. Wang, and X. Xing, "FARE: Enabling fine-grained attack categorization under low-quality labeled data," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–18.
- [32] S. Wang et al., "Fast parameter-free multi-view subspace clustering with consensus anchor guidance," *IEEE Trans. Image Process.*, vol. 31, pp. 556–568, 2021.
- [33] L. Zhang, L. Fu, T. Wang, C. Chen, and C. Zhang, "Mutual information-driven multi-view clustering," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2023, pp. 3268–3277.
- [34] J. Cai and J. Fan, "Perturbation learning based anomaly detection," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 14317–14330.
- [35] Y. Zhang, Y. Sun, J. Cai, and J. Fan, "Deep orthogonal hypersphere compression for anomaly detection," in *Proc. Int. Conf. Learn. Represent.*, 2024, pp. 1–25.
- [36] X. Dong, F. Nie, D. Wu, R. Wang, and X. Li, "Joint structured bipartite graph and row-sparse projection for large-scale feature selection," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, May 8, 2024, doi: 10.1109/TNNLS.2024.3389029.
- [37] G. Liu, H. Zhao, F. Fan, G. Liu, Q. Xu, and S. Nazir, "An enhanced intrusion detection model based on improved kNN in WSNs," *Sensors*, vol. 22, no. 4, p. 1407, Feb. 2022. [Online]. Available: <https://www.mdpi.com/1424-8220/22/4/1407>
- [38] G. Mohi-ud-din et al., "Intrusion detection using hybrid enhanced CSA-PSO and multivariate WLS random-forest technique," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 4, pp. 4937–4950, Dec. 2023.
- [39] J. Azimjonov and T. Kim, "A comprehensive empirical analysis of data sets, regression-based feature selectors, and linear SVM classifiers for intrusion detection systems," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34676–34693, Nov. 2024.
- [40] M. A. Elsayed, M. Wrana, Z. Mansour, K. Lounis, S. H. H. Ding, and M. Zulkernine, "AdaptIDS: Adaptive intrusion detection for mission-critical aerospace vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 23459–23473, Dec. 2022.
- [41] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.
- [42] D. Pujol-Perich, J. Suarez-Varela, A. Cabellos-Aparicio, and P. Barlet-Ros, "Unveiling the potential of graph neural networks for robust intrusion detection," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 49, no. 4, pp. 111–117, Jun. 2022.
- [43] W. Wang, X. Du, D. Shan, R. Qin, and N. Wang, "Cloud intrusion detection method based on stacked contractive auto-encoder and support vector machine," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1634–1646, Jul. 2022.
- [44] A. Telikani et al., "A cost-sensitive machine learning model with multitask learning for intrusion detection in IoT," *IEEE Trans. Ind. Informat.*, vol. 20, no. 3, pp. 3880–3890, Mar. 2024.
- [45] X. Hu, W. Gao, G. Cheng, R. Li, Y. Zhou, and H. Wu, "Toward early and accurate network intrusion detection using graph embedding," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5817–5831, 2023.

- [46] S. Liu et al., "Efficient one-pass multi-view subspace clustering with consensus anchors," in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 7, pp. 7576–7584.
- [47] W. Yang, Y. Wang, C. Tang, H. Tong, A. Wei, and X. Wu, "One step multi-view spectral clustering via joint adaptive graph learning and matrix factorization," *Neurocomputing*, vol. 524, pp. 95–105, Mar. 2023.
- [48] J. Zhu, W. Zhao, H. Yang, and F. Nie, "Joint learning of anchor graph-based fuzzy spectral embedding and fuzzy K-means," *IEEE Trans. Fuzzy Syst.*, vol. 31, no. 11, pp. 4097–4108, Nov. 2023.



Feng Yang received the Ph.D. degree in computer science and technology from Tsinghua University, Beijing, China, in 2007. He is currently a Researcher with the Institute of Systems Engineering, Beijing. His research interests include information systems and network security.



Yixuan Wu received the M.S. degree in electronics and communication engineering from Northwestern Polytechnical University, Xi'an, China, in 2022, where he is currently pursuing the Ph.D. degree. His research interests include network measurement and anomaly detection.



Linru Ma received the B.E. and Ph.D. degrees in communication and information systems from the National University of Defense Technology, Changsha, China, in 2000 and 2007, respectively. She is currently a Researcher with the Institute of Systems Engineering, Beijing, China. Her research interests include cyber security architecture and system design in resilience.



Long Zhang received the B.S. degree in computer science and technology from Tsinghua University, Beijing, China, in 2010, and the M.S. and Ph.D. degrees in electric science and the technology from the National University of Defense Technology, Changsha, China, in 2012 and 2017, respectively. He is currently an Engineer with the National Key Laboratory of Science and Technology on Information System Security, Institute of Systems Engineering, Beijing. His research interests include network security and formal methods.



Zhoumin Lu received the M.S. degree in computer technology from Fuzhou University, China, in 2021. He is currently pursuing the Ph.D. degree with the School of Computer Science, Northwestern Polytechnical University, Xi'an, China. His research interests include machine learning, deep learning, and their applications, such as pattern recognition and data mining.



Lin Yang received the M.S. degree in automation and the Ph.D. degree in communication and electronic system from the National University of Defense Technology, Changsha, China, in 1995 and 1999, respectively. He is currently a Researcher with the School of Electronics and Information, Northwestern Polytechnical University, Xi'an, China. He is also currently a Researcher with the Institute of Systems Engineering, Beijing. His research interests include computer security, information system security, network security, trusted computing, security protocol analysis, and big data security.



Wen Jiang (Member, IEEE) received the bachelor's and master's degrees from Information Engineering University, Zhengzhou, China, in 1994 and 1997, respectively, and the Ph.D. degree from Northwestern Polytechnical University, Xi'an, China, in 2009. She is currently a Professor with the School of Electronics and Information, Northwestern Polytechnical University. Her research interests include information fusion, artificial intelligence, remote sensing image processing, and intelligent algorithm security.